Laboratory Exercise – Search and A*

Using an LLM as an Engineering Assistant

Artificial Intelligence

Contents

1 Learning Objectives

By the end of this laboratory, you should be able to:

1. formulate a simple navigation problem as a search problem;
2. identify states, actions, transitions, initial states, goals, and costs;
3. explain the difference between blind and informed search;
4. describe the role of a heuristic in A* search;
5. use an LLM to assist in implementing a search algorithm;
6. test and validate code generated by an LLM;
7. compare the behaviour of search algorithms using appropriate measures;
8. reflect critically on the use of an LLM as an engineering tool.

2 Background

In the lecture, we introduced search as a general mechanism for solving problems.
A search problem can be represented by

$$\mathcal{P} = (S, A, T, s_0, G, c),$$

where

- S is the set of states;
- A is the set of actions;
- T is the transition function;
- s_0 is the initial state;
- G is the set of goal states;
- c is the cost function.

We then considered several strategies for deciding which state to explore next:

BFS DFS Greedy A*.

For A*, the evaluation function is

$$f(n) = g(n) + h(n),$$

where

$$g(n) = \text{cost from the start to } n$$

and

$$h(n) = \text{estimated cost from } n \text{ to the goal.}$$

The purpose of this laboratory is to implement and experiment with A* search.

You will use an LLM as a software engineering assistant.

Important

The objective is **not** simply to obtain a program that works.
You are expected to understand the problem, specify the algorithm, test the generated program, and explain the results.
An LLM-generated program is a hypothesis about how the solution should be implemented. It must be tested and validated.

3 Software Requirements

You will need:

- Python 3;
- a Python development environment such as VS Code, IDLE, Jupyter, or a terminal;
- access to an LLM.

You may use a freely available LLM such as:

- ChatGPT;
- Gemini;
- Claude;
- Microsoft Copilot.

No specialised AI or machine-learning library is required.

The aim is to implement the search algorithm yourself, with the assistance of an LLM.

4 Scenario: Warehouse Robot Navigation

Imagine a small autonomous robot operating in a warehouse.

The robot must move from a loading area to a delivery area. Some parts of the warehouse are blocked by shelves.

The robot can move one cell at a time:

$$\text{Up}, \quad \text{Down}, \quad \text{Left}, \quad \text{Right}.$$

Every movement has cost 1.

The warehouse is represented by the following ASCII map.

```
##################
#S....#.........#
#.###.#.#######.#
#...#.#.......#.#
###.#.#######.#.#
#...#.........#.#
#.###########.#.#
#............#G#
##################
```

The symbols have the following meanings:

Symbol	Meaning
`#`	Obstacle
`.`	Free cell
`S`	Starting position
`G`	Goal position

Your task is to construct an agent that finds a path from `S` to `G`.

Task 0: Understand the Search Problem

Before writing any code, formulate the warehouse problem as a search problem. Complete the following table.

Component	Your specification
State S	
Actions A	
Transition T	
Initial state s_0	
Goal G	
Cost c	

Then answer:

(a) What information is necessary to specify a state?

(b) What makes an action invalid?

(c) Is this a deterministic search problem?

(d) What would constitute a solution?

Think About It

Do not ask an LLM for code yet.
The purpose of this task is to separate

understanding the problem

from

implementing the solution.

If you cannot clearly specify the search problem, generating code is premature.

Task 1: Plan the Agent

Before using an LLM, write down a short design for your search agent.
Your design should specify:

1. how a state will be represented in Python;
2. how the warehouse will be represented;
3. how valid actions will be determined;
4. how the agent will recognise that it has reached the goal;
5. what information must be stored in the frontier;
6. how the final path will be reconstructed.

You should also decide what information the program should report when it terminates.
At a minimum, record:

- whether a solution was found;
- the solution path;
- the path length;
- the number of states expanded.

Think About It

This task is deliberately performed *before* prompting the LLM.
You are designing the agent.
The LLM is being used later as an engineering tool to help translate your design into code.

Task 2: Ask an LLM to Generate A*

Now use an LLM to generate a Python implementation of A* for the warehouse problem.
You may use ChatGPT, Gemini, Claude, Copilot, or another freely available LLM.

Begin with a prompt based on your own design from Task 2.
For example:

Example prompt
I am implementing a simple goal-based search agent in Python.
The environment is a grid represented by an ASCII map. The agent starts at S and must reach G. The symbols # represent obstacles and . represents free cells.
The agent can move up, down, left, or right, and every movement has cost 1.
Implement A* search.
Use Manhattan distance as the heuristic:

$$h(n) = |x - x_G| + |y - y_G|.$$

The program should:

- represent grid positions as states;
- maintain an appropriate frontier;
- calculate $g(n)$, $h(n)$ and $f(n)$;
- avoid repeatedly expanding the same state;
- reconstruct the path when the goal is reached;
- report the path and its length;
- report the number of states expanded.

Keep the implementation simple and explain the main components of the code.

Run the generated program on the warehouse map.
Do not assume that the generated code is correct.

Task 3: Test the Generated Program

Test the program systematically.
At a minimum, perform the following tests.

Test 1: Original warehouse

Run the program on the map supplied in this laboratory.
Record:

- whether a path was found;
- the path;
- the path length;
- the number of states expanded.

Test 2: Trivial case

Construct a very small map in which the goal is immediately adjacent to the start.

For example:

```
#####
#SG##
#####
```

Check that the program finds the one-step solution.

Test 3: No solution

Construct a map in which the goal is completely inaccessible.
For example:

```
#######
#S....#
###.###
#...#G#
#######
```

Check that the program reports failure rather than entering an infinite loop.

Test 4: Alternative paths

Construct a map containing two or more paths from S to G.
Check whether the returned path is a shortest path.

Think About It

A program that produces a plausible path is not necessarily correct.
Testing should deliberately include cases in which the expected behaviour is known.
This is an important principle of AI engineering:

$$\boxed{\text{Working output} \neq \text{validated algorithm}}$$

Task 4: Inspect the A* Algorithm

Return to the code generated by the LLM.
Identify where each of the following concepts appears in the program:

Concept	Where does it appear in the code?
State	
Action	
Transition	
Goal test	
$g(n)$	
$h(n)$	
$f(n)$	
Frontier	
Visited states	
Path reconstruction	

Then answer:

(a) What data structure is used for the A* frontier?

(b) How does the program select the next state to expand?

(c) Where is the heuristic calculated?

(d) Does the program explicitly calculate $f(n) = g(n) + h(n)$?

(e) How does the program prevent unnecessary repeated exploration?

Task 5: Compare A* with Blind Search

Modify your program, or ask the LLM to produce a BFS version of the same agent.
Do not change the warehouse.
Run both algorithms on the same map.
Record:

Measure	BFS	A*
Solution found		
Path length		
States expanded		

Then answer:

(a) Did both algorithms find a solution?

(b) Did they find paths of the same length?

(c) Which algorithm expanded fewer states?

(d) Why might A* expand fewer states?

Think About It

A* is not necessarily better simply because it is "more intelligent."
Its behaviour depends on the quality of the heuristic.
The important question is:

What information does the algorithm use to decide where to search next?

Task 6: Investigate the Heuristic

The laboratory has used Manhattan distance:

$$h(n) = |x - x_G| + |y - y_G|.$$

Ask the LLM to explain why this heuristic is appropriate for the warehouse when the robot can move only horizontally and vertically.
Then investigate the following questions experimentally:

1. What happens if the heuristic is replaced by $h(n) = 0$?
2. What happens if the heuristic is replaced by Euclidean distance?
3. What happens if the heuristic is multiplied by 2?

For each version, record:

- whether a solution is found;
- the path length;
- the number of states expanded.

Think About It

The lecture introduced admissibility:

$$h(n) \leq h^*(n).$$

Ask yourself:

What happens to the behaviour of A* when the heuristic becomes too optimistic or too aggressive?

Do not simply ask the LLM for the answer. Use your experiments to investigate the question.

Task 7: Evaluate the LLM-Generated Agent

Reflect on the use of the LLM as an engineering assistant.
Answer the following questions.

1. What parts of the generated code were correct immediately?
2. Did you find any bugs or design problems?
3. How did you discover those problems?
4. Did the LLM use terminology or data structures that you did not understand?
5. Did you modify the LLM-generated code?
6. Which tests were most useful?
7. Could you have trusted the program without testing it?
8. What did you understand about A* that you did not understand before implementing it?

Important

Your report should distinguish between:

1. what you designed yourself;
2. what the LLM suggested;
3. what you accepted;
4. what you changed;
5. what you tested.

The purpose of the exercise is to learn how to use an LLM responsibly as an engineering tool.

5 Submission

If you are asked for a submission: submit a short laboratory report containing:

1. your formulation of the search problem;
2. your design of the agent;
3. the final Python program;
4. the prompts used with the LLM;
5. the results of your tests;
6. the BFS/A* comparison;
7. your heuristic investigation;
8. answers to the reflection questions.

Your report should include enough information for another student to reproduce your experiments.

If you use an LLM, include the important prompts in an appendix or provide them as a separate text file.

6 Final Reflection

Answer the following questions in approximately one paragraph each.

1. Why is it important to formulate the search problem before writing the search algorithm?
2. In what sense is A* an "informed" search algorithm?
3. Why does the choice of heuristic matter?
4. What did the LLM contribute to the engineering process?
5. What could go wrong if an engineer simply accepted LLM-generated code without testing it?

The central lesson of this laboratory is:

$$\boxed{\text{AI Science} \quad \longrightarrow \quad \text{AI Engineering}}$$

The scientific idea is search.

A* is an algorithm implementing one approach to that idea.

The LLM is an engineering tool that can help implement the algorithm.

The engineer remains responsible for understanding, testing, and validating the resulting system.